CSS - Complete Assessment-Ready Notes

Start to End | Hinglish | Concept -> Rule -> Code -> Trap -> Solved Questions

Target: Is PDF ko revise karke CSS ke selectors, specificity, box-model, flexbox, positioning aur responsive-layout questions ko independently reason kar pana. Ye notes hamare complete CSS preparation track ko ek connected revision system mein convert karte hain.

> **Important:** Ye notes assessment-oriented hain, lekin kisi specific company paper ko guarantee nahi karte. Focus transferable CSS reasoning par hai: unfamiliar code dekhkar final style/layout predict karna aur missing CSS likhna.

Module	Status	Practice Result
Selectors + Cascade	Mastered	15/15
Specificity + Box Model	Mastered	18/18
Display + Flexbox	Mastered	20/20
Positioning	Mastered	18/18
Responsive CSS	Mastered	20/20
Final Mixed CSS	Mastered	20/20

How to Use This PDF

1. First pass: concepts + examples padho; formulas/traps ko highlight mentally karo.
2. Second pass: har code block ka output/layout khud predict karo before reading the explanation.
3. Third pass: Assessment Decision Flow aur Final Solved Test ko bina notes dekhe attempt karo.
4. Last-minute revision: Master Tables + 40 High-Value Traps + Rapid Revision Sheet padho.

Complete Module Map

Module	Topics
1. CSS Fundamentals	syntax, selector/property/value, element/class/id/universal/grouping
2. Selectors	descendant, child, multiple classes, attribute, pseudo-class, pseudo-element
3. Cascade + Specificity	specificity tuples, source order, inline, !important basics, inheritance
4. Box Model	content, padding, border, margin, content-box, border-box, shorthand, calculations
5. Display	block, inline, inline-block, none, visibility:hidden
6. Flexbox	container/items, axes, justify, align, gap, wrap, flex:1, order, practical layouts
7. Positioning	static, relative, absolute, fixed, sticky, positioned ancestor, z-index, overflow
8. Responsive CSS	%, vw, vh, max-width, images, media queries, mobile-first, meta viewport
9. Practical Patterns	navbar, card row, badge, centered box, floating button, mobile stacking
10. Assessment Traps	specificity + media, direct child, flex direct child, flow removal, box math
11. Solved Final Test	20 mixed assessment-style questions with reasoning
12. Rapid Revision	one-shot tables, formulas and decision flow

Module 1 - CSS Fundamentals

CSS ka kaam HTML structure ko presentation/style dena hai. Browser ko teen cheezein chahiye: kis element ko select karna hai, kaunsi property change karni hai, aur us property ki value kya hogi.

```
p {
    color: red;
}
```

Part	Meaning
p	Selector - kis element ko target karna hai
color	Property - kya change karna hai
red	Value - property ko kya value deni hai

> **Master Syntax:** selector { property: value; }

1.1 Element Selector

```
p {
    color: blue;
}
```

Saare matching <p> elements select honge.

1.2 Class Selector

```
.message {
    color: green;
}
```

Dot (.) class ko represent karta hai. Ek class multiple elements par reuse ho sakti hai.

1.3 ID Selector

```
#title {
    color: red;
}
```

Hash (#) id ko represent karta hai. HTML id document mein unique design ki taraf use hota hai.

1.4 Universal Selector

```
* {
    box-sizing: border-box;
}
```

Universal selector broadly all elements ko match karta hai.

1.5 Grouping Selector

```
h1,
h2,
p {
    color: blue;
}
```

Comma ka matlab: har listed selector par same declaration block apply karo.

Module 2 - Selectors and Combinators

2.1 Descendant Selector - Space

```
.card p {
    color: red;
}
```

.card ke andar kisi bhi depth par jo <p> descendant ho, wo match karega.

```
<div class="card">
    <p>A</p>
    <section>
        <p>B</p>
    </section>
</div>
```

Result: A aur B dono match.

2.2 Direct Child Selector - >

```
.card > p {
    color: red;
}
```

Sirf .card ke direct child <p> ko select karega. Nested section ke andar ka <p> match nahi karega.

> **Trap:** A B ka matlab descendant; A > B ka matlab direct child.

2.3 Multiple Classes on the Same Element

```
<p class="note active">Hello</p>
```

```
.note.active {
    color: red;
}
```

Space nahi hai: same element ke paas note AND active dono classes honi chahiye.

2.4 Same Element vs Descendant - Killer Difference

Selector	Meaning	Typical HTML
.card.active	Same element has both classes	
.card .active	An .active descendant inside .card	

2.5 Attribute Selector

```
input[type="password"] {
```

```
    border: 2px solid red;
}
```

Sirf password-type input select hoga.

2.6 Pseudo-class

```
button:hover {
    background-color: black;
}

input:focus {
    border: 2px solid blue;
}
```

Pseudo-class element ki state/condition ko target karta hai. Common: :hover, :focus, :checked, :first-child, :last-child.

2.7 Pseudo-element

```
p::first-letter {
    font-size: 30px;
}

p::before {
    content: "> ";
}
```

Pseudo-element element ke specific part ya generated content ko style karta hai.

> **Lock:** :hover = pseudo-class; ::before / ::first-letter = pseudo-element.

Module 3 - Cascade, Specificity and Inheritance

Ek element par multiple CSS rules simultaneously match kar sakte hain. Browser ko winner decide karna hota hai. Isi decision system ka major part cascade + specificity + source order hai.

3.1 Simplified Specificity Tuple

Assessment ke liye hum tuple use karte hain:

```
(ID, class, element)
```

Selector Type	Contribution
#id	(1,0,0)
.class / [attribute] / :pseudo-class	(0,1,0)
element / ::pseudo-element	(0,0,1)

Tuple ko left-to-right compare karo. Normal decimal addition ki tarah mat socho.

3.2 Examples

Selector	Specificity
p	(0,0,1)
.note	(0,1,0)
#title	(1,0,0)
.card p	(0,1,1)
.card.active	(0,2,0)
p.note.active	(0,2,1)
#app .card.active p	(1,2,1)

3.3 Specificity Winner

```
<p id="msg" class="note active">Hello</p>
```

```
.note.active {
    color: green;
}

#msg {
    color: red;
}

p.note.active {
    color: blue;
}
```

Specificity: .note.active=(0,2,0), #msg=(1,0,0), p.note.active=(0,2,1). Final color: red because ID wins.

3.4 Same Specificity -> Source Order

```
.note {
    color: red;
}

.note {
    color: blue;
}
```

Same specificity, same cascade level: later declaration wins -> blue.

> **Critical:** Source order tab decide karta hai jab competing rules ki priority/specificity tied ho. Lower-specificity rule sirf baad mein likha hone se higher-specificity rule ko nahi hara deta.

3.5 Inline Style

```
<p id="message" style="color: purple;">Hello</p>
```

```
#message {
    color: red;
}
```

Normal author-style case mein inline declaration stronger hoti hai -> purple.

3.6 !important - Basic Assessment Rule

```
p {
    color: red !important;
}

#title {
    color: blue;
}
```

Basic assessment mental model: !important normal declaration priority ko change karta hai. Real CSS cascade origins/layers aur stacking of important declarations more nuanced ho sakte hain, lekin basic MCQ mein important declaration ko normal declaration se stronger samjho.

3.7 CSS Inheritance

```
.parent {
    color: green;
}
```

```
<div class="parent">
    <p>Hello</p>
</div>
```

Agar <p> par competing color rule nahi hai, text green inherit karega.

Often Inherited	Usually Not Inherited
color	margin
font-family	padding
font-size	border
	width / height

Do not confuse: CSS inheritance Java inheritance nahi hai. Yahan parent element se computed style property child tak propagate ho sakti hai.

Module 4 - Box Model and Dimension Calculations

Har element ko ek rectangular box ki tarah socho.

```
Inside -> Outside
Content -> Padding -> Border -> Margin
```

4.1 Padding vs Margin

Part	Meaning
Content	Actual content area
Padding	Content aur border ke beech internal space
Border	Padding/content ko surround karta hai
Margin	Border ke bahar external space

> **Background Trap:** Margin element ke background color ka normal part nahi hota. Padding element ke visual/background area ke andar hoti hai.

4.2 content-box - Default Mental Model

```
.box {
    width: 200px;
    padding: 20px;
    border: 5px solid black;
}
```

Default content-box mein declared width content ki width hoti hai.

```
Outer border-box width
= content width
+ left padding + right padding
+ left border + right border

= 200 + 20 + 20 + 5 + 5
= 250px
```

4.3 Margin Included Occupied Width

```
.box {
    width: 200px;
    padding: 20px;
    border: 5px solid black;
    margin: 10px;
}
```

```
Border-box width = 250px
Total horizontal occupied space
= 250 + 10 + 10
```

```
= 270px
```

4.4 border-box

```
.box {
    box-sizing: border-box;
    width: 300px;
    padding: 25px;
    border: 5px solid black;
}
```

Outer border-box width exactly 300px. Content width:

```
300 - 25 - 25 - 5 - 5 = 240px
```

> **Master Difference:** content-box: declared width = content only. border-box: declared width includes content + padding + border.

4.5 Common Reset

```
* {
    box-sizing: border-box;
}
```

Sizing predictable ho jati hai because declared dimensions ke andar padding/border fit hote hain.

4.6 Shorthand Rules

CSS	Meaning
padding: 20px;	all four sides = 20
padding: 10px 20px;	top/bottom=10, left/right=20
padding: 10px 20px 30px;	top=10, left/right=20, bottom=30
padding: 10px 20px 30px 40px;	top=10, right=20, bottom=30, left=40

> **Memory:** 4-value shorthand = TRBL = Top Right Bottom Left.

4.7 Width Calculation Example

```
.box {
    width: 300px;
    padding: 10px 20px;
    border: 5px solid black;
    margin: 15px;
}
```

```
Horizontal padding = 20 + 20 = 40
Horizontal border = 5 + 5 = 10
Border-box width = 300 + 40 + 10 = 350px
Including margins = 350 + 15 + 15 = 380px
```

4.8 Height Calculation

```
.box {
    height: 100px;
    padding: 10px;
    border: 2px solid black;
}
```

```
Outer height = 100 + 10 + 10 + 2 + 2 = 124px
```

Module 5 - Display

Value	Core Behavior
block	New line; block formatting; width/height usable
inline	Same line; width/height do not behave like normal block dimensions
inline-block	Same line + width/height control
none	Element hidden and removed from layout

5.1 display:none vs visibility:hidden

Rule	Visible?	Layout Space?
display: none	No	No
visibility: hidden	No	Yes

> **Common MCQ:** display:none removes layout participation; visibility:hidden preserves space.

5.2 Inline vs Inline-block

```
<span>A</span>
<span>B</span>
```

Spans normally same line par aa sakte hain. Agar same line + dimensions dono chahiye, inline-block use karo.

Module 6 - Flexbox

Flexbox one-dimensional layout system hai jo direct child items ko row/column mein align, distribute aur wrap karne ke liye bahut useful hai.

6.1 Flex Container and Flex Items

```
<div class="container">
    <div>A</div>
    <div>B</div>
    <div>C</div>
</div>
```

```
.container {
    display: flex;
}
```

.container = flex container. Iske direct children A/B/C = flex items.

> **Nested Trap:** Grandchildren automatically outer flex container ke direct flex items nahi bante.

6.2 Default Direction

```
flex-direction: row;
```

Default main axis horizontal hota hai.

6.3 Main Axis and Cross Axis

Direction	Main Axis	Cross Axis
row	Horizontal	Vertical
column	Vertical	Horizontal

> **Ultimate Flex Rule:** justify-content -> MAIN axis. align-items -> CROSS axis.

6.4 justify-content

Value	Basic Effect
flex-start	items main-axis start side
flex-end	items main-axis end side
center	items main-axis center
space-between	large equal space between items; first/last toward edges
space-around	space around items; outer edge gap smaller-looking

space-evenly	all gaps including outer edges equal

6.5 align-items

Cross-axis alignment control karta hai. Default row mein align-items:center means vertical centering. Column mein cross-axis horizontal ho jata hai.

6.6 Perfect Centering

```
.container {
    height: 300px;
    display: flex;
    justify-content: center;
    align-items: center;
}
```

6.7 gap

```
.container {
    display: flex;
    gap: 20px;
}
```

Flex items ke beech consistent gap.

6.8 flex-wrap

```
flex-wrap: wrap;
```

Available main-axis space kam ho to flex items next flex line par ja sakte hain.

6.9 flex: 1

```
.item {
    flex: 1;
}
```

Common assessment shortcut: sibling flex items available space ko roughly equally share/grow kar sakte hain.

6.10 order

```
.a { order: 2; }
.b { order: 1; }
```

Visual order change ho sakta hai. Real UI accessibility ke liye DOM order still important hai.

6.11 Practical Navbar

```
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
```

```
}
```

Logo beginning ke paas, menu end ke paas, dono cross-axis par aligned.

Module 7 - Positioning, z-index and Overflow

position	Normal Flow?	Reference / Behavior
static	Yes	Default normal flow
relative	Yes	Own original position se offset; original space retained
absolute	No	Nearest positioned (non-static) ancestor
fixed	No	Viewport-relative; scroll par same screen position
sticky	Initially yes	Threshold/offset hit hone par scroll container/viewport edge ke paas stick

7.1 relative

```
.box {
    position: relative;
    left: 20px;
    top: 10px;
}
```

Element visual position shift karta hai, lekin original document-flow space retain hoti hai.

7.2 absolute

Absolute element normal flow se remove ho jata hai. top/right/bottom/left nearest positioned ancestor ke relative calculate hote hain.

7.3 Badge Pattern - Most Important Practical

```
<div class="card">
    Product
    <span class="badge">NEW</span>
</div>
```

```
.card {
    position: relative;
}

.badge {
    position: absolute;
    top: 0;
    right: 0;
}
```

> **Why parent relative?:** Parent ko visually move karna zaroori nahi. position:relative often sirf absolute child ka positioning reference establish karne ke liye use hota hai.

7.4 Nearest Positioned Ancestor

```
.outer { position: relative; }
.inner { position: static; }
.badge { position: absolute; }
```

Badge .inner ko skip karega because static. Reference .outer hoga. Agar .inner bhi relative ho jaye, nearest .inner reference banega.

7.5 fixed

```
.chat {
    position: fixed;
    right: 20px;
    bottom: 20px;
}
```

Floating chat/action button viewport ke bottom-right mein stay kar sakta hai even during scroll.

7.6 sticky

```
.header {
    position: sticky;
    top: 0;
}
```

Initially normal flow mein; scroll threshold hit karne par top ke paas stick. Sticky ancestor overflow/scroll context se affect ho sakta hai.

7.7 z-index

```
.a { position: absolute; z-index: 2; }
.b { position: absolute; z-index: 10; }
```

Simplified overlapping case: higher z-index front par -> .b. Real stacking contexts more nuanced ho sakte hain.

7.8 overflow

Value	Basic Behavior
visible	overflow box ke outside visible ho sakta hai
hidden	overflow clip/hide
scroll	scrolling mechanism/scrollbars
auto	scrolling only when needed

Module 8 - Responsive CSS and Media Queries

Responsive design ka goal same page ko different screen sizes par usable aur properly arranged rakhna hai.

8.1 % vs vw vs vh

Unit	Reference
%	Usually containing block / parent dimension
vw	Viewport width
vh	Viewport height

```
Viewport width = 1200px
width: 25vw -> 300px

Parent width = 800px
width: 25% -> 200px
```

8.2 width + max-width

```
.container {
    width: 90%;
    max-width: 1200px;
}
```

Container normally available parent width ka 90% lega, lekin 1200px se wider nahi hoga.

8.3 Responsive Images

```
img {
    max-width: 100%;
    height: auto;
}
```

Image container se wider hone se bachegi; height auto aspect ratio maintain karne mein help karta hai.

8.4 Media Query - max-width

```
@media (max-width: 600px) {
    .menu {
        display: none;
    }
}
```

Condition width 600px ya kam par active.

8.5 Media Query - min-width

```
@media (min-width: 768px) {
    .container {
        flex-direction: row;
    }
```

```
}
```

768px ya upar active. Mobile-first approach mein base styles small screens ke liye aur min-width queries larger screens ke enhancements ke liye common hain.

8.6 Desktop Row -> Mobile Column

```
.container {
    display: flex;
    flex-direction: row;
}

@media (max-width: 600px) {
    .container {
        flex-direction: column;
    }
}
```

8.7 Media Query Does Not Increase Specificity

```
#text {
    color: red;
}

@media (max-width: 600px) {
    .note {
        color: blue;
    }
}
```

Viewport 500px par media query active hai, but #text specificity higher hai -> red.

8.8 Same Specificity + Active Media + Source Order

```
.note {
    color: red;
}

@media (max-width: 600px) {
    .note {
        color: blue;
    }
}
```

Viewport 500px: both .note same specificity; later active rule -> blue.

8.9 Meta Viewport

```
<meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
>
```

Mobile browser ko page viewport device width ke according handle karne mein help karta hai.

8.10 flex-basis - Basic Note

```
.card {
    flex: 1 1 250px;
}
```

Basic interpretation: grow allowed, shrink allowed, preferred initial main-axis size about 250px. Responsive wrapping layouts mein useful.

Module 9 - Practical CSS Patterns You Should Write Without Help

9.1 Horizontal + Vertical Center

```
.container {
    height: 300px;
    display: flex;
    justify-content: center;
    align-items: center;
}
```

9.2 Two Ends Toolbar

```
.toolbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
}
```

9.3 Card Badge at Top-right

```
.card {
    position: relative;
}

.badge {
    position: absolute;
    top: 0;
    right: 0;
}
```

9.4 Floating Chat Button

```
.chat {
    position: fixed;
    right: 20px;
    bottom: 20px;
}
```

9.5 Responsive Card Row -> Column

```
.cards {
    display: flex;
    gap: 20px;
}

@media (max-width: 600px) {
    .cards {
        flex-direction: column;
    }
}
```

9.6 Full Combined Pattern

```
.cards {
    display: flex;
    gap: 20px;
    flex-direction: row;
}

.card {
    position: relative;
}

.badge {
    position: absolute;
    top: 0;
    right: 0;
}

@media (max-width: 600px) {
    .cards {
        flex-direction: column;
    }
}
```

Is pattern mein Flexbox + positioning + responsive media query ek saath combine hote hain. Assessment practical tasks ke liye high-value pattern hai.

Module 10 - High-Value Assessment Traps

#	Trap	Correct Rule
1	.note.active	Same element has both classes.
2	.note .active	active descendant inside note; same element nahi.
3	.box > p	Only direct child p.
4	.box p	Any descendant p.
5	Same specificity	Later rule wins only after cascade priority ties.
6	#id vs later .class	ID specificity wins in normal case.
7	Media query	Condition active hone se selector ki specificity increase nahi hoti.
8	content-box	Declared width content only; padding/border add outside.
9	border-box	Declared width includes padding + border.
10	margin	Background area ka normal part nahi.
11	inline	Same line; width/height block-style control limited.
12	inline-block	Same line + width/height.
13	display:none	Hidden + no layout space.
14	visibility:hidden	Hidden + layout space retained.
15	Flex direct child	Only container ke direct children flex items.
16	justify-content	MAIN axis - not permanently horizontal.
17	align-items	CROSS axis - not permanently vertical.
18	column direction	Main vertical, cross horizontal.
19	space-between	First/last toward edges, space between.
20	space-evenly	All gaps including outer edges equal.
21	flex-wrap:wrap	Items next flex line par ja sakte hain.
22	position:relative	Normal flow remains; original space retained.

23	position:absolute	Removed from normal flow.
24	absolute reference	Nearest non-static positioned ancestor.
25	parent relative	Often absolute child ke containing reference ke liye.
26	position:fixed	Viewport-relative, normal flow removed.
27	position:sticky	Normal flow initially, threshold/offset par sticks.
28	z-index	Simplified: higher overlapping z-index appears above.
29	overflow:hidden	Overflow clipped.
30	overflow:auto	Scrolling when needed.
31	%	Usually containing block relative.
32	vw	Viewport width relative.
33	vh	Viewport height relative.
34	max-width query	Breakpoint and below.
35	min-width query	Breakpoint and above.
36	max-width:100% image	Prevents image from exceeding container width.
37	height:auto image	Helps preserve aspect ratio.
38	max-width property	Caps size; does not force exact width.
39	CSS inheritance	Only inheritable properties; margin/padding do not normally inherit.
40	Output strategy	Never guess visual result; first determine selector match, then cascade, then layout model.

Module 11 - 15-Second CSS Assessment Decision Flow

```
QUESTION DEKHO
    |
    v
1. Kaunse selectors actually element ko match karte hain?
    |
    v
2. Specificity tuple calculate karo.
    |
    v
3. Tie hai? Source order check karo.
    |
    v
4. Property inherited hai ya direct declaration?
    |
    v
5. Box model hai? content-box ya border-box identify karo.
    |
    v
6. Display/flex hai? Direct flex items identify karo.
    |
    v
7. Flex axes determine karo from flex-direction.
    |
    v
8. Positioning hai? Normal flow + containing ancestor identify karo.
    |
    v
9. Media query condition true hai?
    |
    v
10. Responsive rule active hone ke baad cascade dobara apply karo.
    |
    v
11. Exact final style/layout/output likho.
```

Golden Rule: CSS question ko visual guess se solve mat karo. Selector match -> cascade -> layout system -> responsive condition ke fixed order mein trace karo.

Module 12 - Solved Final Mixed CSS Assessment

Ye wahi mixed level hai jahan topics intentionally mix hote hain. Har solution mein final answer ke saath decision rule diya gaya hai.

Q1 - Selector trap

```
HTML:
<div class="card">
    <p class="note active">Hello</p>
</div>

Which selector matches the p having both classes?
A. .note .active
B. .note.active
C. #note.active
D. p > .note
```

Solution: Answer: B. .note.active. Reason: no space means same element must contain both classes.

Q2 - Direct child

```
.box > p { color: red; }

<div class="box">
    <p>A</p>
    <section><p>B</p></section>
</div>
```

Solution: Answer: A = red, B = not red. > selects only direct children.

Q3 - Specificity

```
#app .card.active p
```

Solution: Answer: (1,2,1) -> 1 ID, 2 classes, 1 element.

Q4 - Specificity winner

```
.note.active { color: green; }
#msg { color: red; }
p.note.active { color: blue; }
```

Solution: Specificities: (0,2,0), (1,0,0), (0,2,1). Final: red because #msg wins.

Q5 - Same specificity

```
.card .title { color: red; }
.card .title { color: blue; }
```

Solution: Final: blue. Same specificity; later matching declaration wins.

Q6 - content-box width

```
width: 240px;
padding: 10px 20px;
border: 4px solid black;
```

Solution: Border-box width = 240 + 20 + 20 + 4 + 4 = 288px.

Q7 - margin included

```
width: 240px;
padding: 10px 20px;
border: 4px solid black;
margin: 15px;
```

Solution: Border-box = 288px. Total horizontal occupied = 288 + 15 + 15 = 318px.

Q8 - border-box content width

```
box-sizing: border-box;
width: 300px;
padding: 25px;
border: 5px solid black;
```

Solution: Outer width = 300px. Content width = 300 - 25 - 25 - 5 - 5 = 240px.

Q9 - Display

```
inline / inline-block / none
```

Solution: inline = same line, limited block dimension behavior; inline-block = same line + width/height; none = hidden + removed from layout.

Q10 - Flex axes

```
display:flex;
```

```
flex-direction:column;
justify-content:space-between;
align-items:center;
```

Solution: Main axis = vertical. Cross axis = horizontal. justify-content -> main; align-items -> cross.

Q11 - Toolbar

```
Back at start, Save at end, vertically centered.
```

Solution:

```
.toolbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
}
```

Q12 - Direct flex items

```
<div class="container">
  <div class="card"><span>A</span><span>B</span></div>
  <div class="card">C</div>
</div>
.container { display:flex; }
```

Solution: Direct flex items = 2 card divs. A/B spans are not outer container flex items.

Q13 - Badge positioning

```
SALE at product top-right.
```

Solution:

```
.product { position: relative; }
.sale { position: absolute; top: 0; right: 0; }
```

Q14 - Positioned ancestor

```
.outer { position:relative; }
.middle { position:static; }
.inner { position:relative; }
.badge { position:absolute; top:0; right:0; }
```

Solution: Badge positions relative to .inner because it is the nearest non-static positioned ancestor.

Q15 - Flow

```
relative vs absolute
```

Solution: relative retains normal-flow/original layout space; absolute is removed from normal flow and does not reserve original space.

Q16 - Fixed vs sticky

```
Chat floating at viewport bottom-right; table header sticks after scrolling.
```

Solution: Chat -> fixed. Table header -> sticky.

Q17 - Responsive units

```
Viewport=1200px, parent=800px. 25vw? 25%?
```

Solution: 25vw = 300px. 25% = 200px.

Q18 - Media + source order

```
.card { color:red; }
@media (max-width:700px) { .card { color:blue; } }
.card { color:green; }
Viewport=600px
```

Solution: Final green. Media is active, but all selectors have same specificity and the final green declaration is last.

Q19 - Media + specificity

```
#panel { display:block; }
@media (max-width:500px) { .panel { display:none; } }
Viewport=400px
```

Solution: Visible. Media rule is active but #panel (1,0,0) beats .panel (0,1,0).

Q20 - Final practical

```
Desktop cards row with 20px gap; badge top-right; <=600px cards vertical.
```

Solution:

```
.cards {
    display: flex;
    gap: 20px;
    flex-direction: row;
}

.card {
    position: relative;
}

.badge {
    position: absolute;
    top: 0;
    right: 0;
}

@media (max-width: 600px) {
    .cards {
        flex-direction: column;
    }
}
```

Module 13 - Rapid Revision Sheet

Concept	One-line Rule
Element selector	p -> all matching p elements
Class selector	.note -> class=note
ID selector	#title -> id=title
Descendant	.box p -> any p inside box
Direct child	.box > p -> direct p only
Same element multiple classes	.note.active
Specificity	ID > class/attribute/pseudo-class > element/pseudo-element
Tie	Later source order wins
Box order	Content -> Padding -> Border -> Margin
content-box	width = content only
border-box	width includes padding + border
block	new line + dimensions
inline	same line; limited width/height behavior
inline-block	same line + dimensions
display:none	hidden + no layout space
visibility:hidden	hidden + space retained
Flex direct items	Direct children only
justify-content	main axis
align-items	cross axis
row	main horizontal, cross vertical
column	main vertical, cross horizontal
space-between	space between items, ends toward edges
space-evenly	all gaps equal
wrap	next flex line when needed
relative	flow retained, own-position offset
absolute	flow removed, nearest positioned ancestor

fixed	viewport relative
sticky	threshold par sticks
%	containing block relative
vw	viewport width relative
vh	viewport height relative
max-width media	breakpoint and below
min-width media	breakpoint and above
Media specificity	media query itself adds no specificity

Last 2-Minute Formula Box

```
Specificity:
#app .card.active p = (1,2,1)

content-box outer width:
content + L/R padding + L/R border

occupied width with margins:
border-box width + L/R margin

border-box content width:
declared width - L/R padding - L/R border

Flex:
justify-content -> MAIN axis
align-items -> CROSS axis

row    -> main horizontal, cross vertical
column -> main vertical, cross horizontal

Absolute positioning:
nearest non-static positioned ancestor

Responsive:
% -> container/parent
vw -> viewport width
vh -> viewport height
```

Self-Check Before You Call CSS Ready

- Can you distinguish .card.active from .card .active instantly?
- Can you calculate specificity without guessing?
- Can you calculate content-box and border-box widths?
- Can you switch justify/align correctly when flex-direction becomes column?
- Can you identify only direct flex items?
- Can you explain why absolute removes normal-flow space but relative does not?
- Can you identify the nearest positioned ancestor?
- Can you combine Flexbox + absolute badge + media query in one practical task?
- Can you explain why an active media query may still lose due to specificity?
- Can you reason through the final result before writing the answer?

If all 10 are yes: CSS foundation is assessment-ready. Next high-value step is JavaScript execution tracing + DOM/event handling, then mixed HTML/CSS/JS practical questions.

CSS Complete

Selectors -> Cascade -> Specificity -> Box Model -> Display -> Flexbox -> Positioning -> Responsive CSS

Final memory: CSS ko rule-by-rule ratne ke bajay ek execution system ki tarah dekho. Pehle selector match, phir cascade/specificity, phir box/layout model, phir positioning, phir responsive condition. Isi order se unfamiliar questions manageable ho jate hain.